

Design Your Personal Agent

Ahmed Elgharably — FlyRank ML Internship — General AI Fluency, Week 5 (FL-06)

Agent Name

Capstone Scout — A research assistant that finds, reads, summarizes, and files ML papers relevant to my FlyRank capstone, without me hunting URLs or copy-pasting between tools.

1. Job to Be Done

Primary job: Keep me current on ML research relevant to content refresh scoring, ranking models, CTR prediction, and search intelligence — with zero manual hunting.

What it does end-to-end:

1. Searches arXiv and Semantic Scholar for new papers matching my topics
2. Filters out papers I've already read (by comparing against my existing notes)
3. Downloads the PDF and extracts key sections
4. Synthesizes a structured summary using my research note format
5. Saves the note to my `research-notes/` folder in the correct markdown structure
6. Generates a Monday morning digest: "3 new papers processed. 1 directly relevant to Lane 2. Here's why."

What it does NOT do:

- Write original research or claim novel findings
 - Submit papers to conferences or post to social media
 - Spend money on API calls without my approval
 - Replace my judgment on whether a paper is worth my time
-

2. The User (Me) and Usage Frequency

Who I am: ML Engineering intern at FlyRank, working on content refresh scoring (Lane 2). I need to stay current on ranking models, CTR prediction, content optimization, and embedding-based search. I have ~5 hours per week for reading and note-taking.

Current pain: I spend 60–90 minutes per week just finding papers. Another 30–45 minutes per paper taking notes. Most of that time is mechanical: opening arXiv, skimming abstracts, checking if I already read it, downloading, scrolling for numbers.

Usage frequency:

- **Automatic:** Runs every Monday at 9 AM (scheduled)
- **On-demand:** I can trigger it mid-week if I hear about a specific paper

- **Review time:** I spend 10–15 minutes on Monday morning reading the digest, not 90 minutes hunting

Success metric: I go from “0 papers read this week because I was busy” to “3 papers summarized and ready for review every Monday” without increasing my time investment.

3. Tools and Data Needed — With Access Plan

Tools

Tool	Purpose	How the agent accesses it	Free?
arXiv API	Search for new papers by keyword	HTTP GET requests to <code>export.arxiv.org/api/query</code>	✅ Yes
Semantic Scholar API	Alternative search + citation data	HTTP GET requests to <code>api.semanticscholar.org</code>	✅ Yes (free tier)
PyPDF / pdf-plumber	Extract text from downloaded PDFs	Local Python execution	✅ Yes
Claude API	Synthesize summaries and generate takeaways	API calls with structured prompts	⚠️ Paid (but I use the free tier + existing Claude Pro subscription)
Filesystem MCP	Read existing notes + write new notes	Local filesystem via MCP server	✅ Yes

Data Sources

Data	Location	Access Plan
Existing research notes	<code>~/flyrank-ml-internship/research-notes/*.md</code>	Filesystem MCP server reads these to check for duplicates
My capstone lane description	<code>~/flyrank-ml-internship/docs/ml-intern-dataset-and-lane-guide.md</code>	Filesystem MCP reads this to ground relevance scoring
Identity kit / voice card	Stored in Claude Project instructions	Loaded as system prompt context
Paper PDFs	Downloaded to <code>~/flyrank-ml-internship/papers/</code>	Agent downloads via HTTP, saves locally

Access Plan Detail

arXiv API: No API key needed for basic queries. Rate limit: 1 request per 3 seconds. The agent will batch searches and respect the limit.

Semantic Scholar API: Free tier allows 100 requests/5 minutes. I’ll register for an API key. Fallback: if rate-limited, the agent skips S2 and uses arXiv only.

Claude API: I have a Claude Pro subscription. The agent will use the API for synthesis only (not for search or download). Estimated cost: ~\$0.50 per week (3 papers × ~3K tokens each). If I hit limits, the agent falls back to local summarization with a smaller open model.

Filesystem MCP: Already configured from FL-05. The server has read/write access to ~/flyrank-ml-internship/. No additional setup.

4. Draft Instructions (System Prompt)

You are Capstone Scout, a research assistant for an ML Engineering intern working on content refresh
↪ scoring at FlyRank.

YOUR JOB:

1. Search arXiv and Semantic Scholar for papers published in the last 7 days matching these topics:
 - "content refresh" OR "content optimization" OR "SEO machine learning"
 - "learning to rank" OR "ranking model" OR "search ranking"
 - "CTR prediction" OR "click-through rate" OR "engagement prediction"
 - "content embedding" OR "document embedding" OR "semantic search"

2. For each paper found:
 - a. Check if it already exists in ~/flyrank-ml-internship/research-notes/ by comparing titles.
 - b. If new, download the PDF to ~/flyrank-ml-internship/papers/YYYY-MM-DD-title.pdf
 - c. Extract: title, authors, abstract, publication date, key dataset, main method, best metric,
↪ baseline comparison, limitations, and one practical takeaway for content refresh scoring.

3. Format each paper into a markdown note using this exact structure:

```
# [Title]
**Authors:** [names]
**Date:** [date]
**Source:** [arXiv / Semantic Scholar]

## One-Line Problem
[Core problem in one sentence]

## Data & Scale
[Dataset and size]

## Method
[Main architecture or approach]

## Key Numbers
[Metric table]

## Honest Caveat
[Biggest limitation]

## FlyRank Takeaway
[Practical connection to content refresh scoring]
```

Scouted by Capstone Scout • [date]

```
4. Save the note to ~/flyrank-ml-internship/research-notes/YYYY-MM-DD-title.md

5. Generate a Monday digest:
  "This week: [N] new papers found. [M] directly relevant to Lane 2 (refresh scoring). [K] relevant to
  ↳ related lanes. [L] skipped (already read or off-topic)."
  Then list each paper with one sentence on why it matters.

VOICE RULES:
- Direct, warm, evidence-first, no buzzwords.
- If a number seems suspicious (too good, no baseline, tiny dataset), flag it with ⚠️.
- Never claim the paper "proves" anything causal unless the authors ran an experiment.
- If the FlyRank Takeaway feels forced, say honestly: "No direct connection to refresh scoring, but the
  ↳ method could be adapted."

GUARDRAILS:
- NEVER spend money on API calls without logging the cost estimate first.
- NEVER overwrite an existing note. If a paper is already filed, skip it.
- NEVER download papers larger than 50MB (skip and flag).
- ALWAYS verify the paper was actually published in the last 7 days before including it.
- ALWAYS flag papers behind paywalls instead of trying to bypass them.
```

5. Five Eval Cases (Pre-Build)

These are the scenarios I'll test before declaring the agent "working." Each one must pass for the agent to ship.

Eval Case 1: Finds a Relevant Paper I Haven't Read

Input: Monday morning trigger runs. arXiv has a new paper: "Temporal Content Refresh Scheduling for Search Engines" published 2 days ago. I have no note with this title. **Expected output:** Agent downloads PDF, extracts key info, generates a structured note, saves it to `research-notes/`, and includes it in the digest as "directly relevant to Lane 2." **Pass criteria:** Note exists in folder. Digest mentions it. No duplicate created.

Eval Case 2: Skips a Paper I've Already Read

Input: arXiv returns "BERT: Pre-training of Deep Bidirectional Transformers" (which I already have a note for from FL-04). **Expected output:** Agent compares titles, finds the match, skips download and synthesis, and reports in the digest: "1 paper skipped (already read)." **Pass criteria:** No duplicate note created. Digest accurately reports the skip.

Eval Case 3: Handles a Paper with No Empirical Results

Input: Agent finds a purely theoretical paper: "Axiomatic Foundations of Ranking Fairness" with no dataset, no experiments, no numbers. **Expected output:** Agent flags it in the digest: "1 theoretical paper found — no empirical results, saved as 'skim-only' note." **Pass criteria:** Note is created but marked as theoretical. Agent doesn't invent fake numbers.

Eval Case 4: Respects Rate Limits and Handles Failure

Input: arXiv API returns 503 (service unavailable). Semantic Scholar API returns rate limit error. **Expected output:** Agent waits 10 seconds, retries arXiv once. If still failing, generates a digest saying: “Search services temporarily unavailable. No new papers this week. Try again tomorrow.” Does not crash or loop infinitely. **Pass criteria:** Graceful degradation. Digest sent. No infinite retries.

Eval Case 5: Produces a Digest I Can Read in 2 Minutes

Input: Agent found 5 new papers, processed 3, skipped 2 (already read). **Expected output:** A digest under 300 words that I can read in 2 minutes. It tells me: how many papers, which one is most relevant to my capstone, and what action I should take (“Read the first one — it directly addresses refresh scheduling”). **Pass criteria:** Digest is 300 words. I can decide which paper to read first without opening any notes.

6. Risks and Guardrails

Risk 1: Hallucinated Paper Summaries

What could go wrong: The agent invents a metric or misrepresents the method. I trust the note, cite it in my capstone, and look foolish. **Guardrail:** Every note includes a “VERIFY” section with the exact page numbers where key numbers appear. I spot-check 1 in 3 notes against the PDF. The agent never claims to have “read” the paper — it claims to have “extracted” text, and the burden of verification stays with me.

Risk 2: Duplicate Notes with Slightly Different Titles

What could go wrong: arXiv lists “Learning to Rank: A Survey” and Semantic Scholar lists “A Survey on Learning to Rank.” The agent treats them as different papers and creates two notes. **Guardrail:** The agent normalizes titles (lowercase, remove punctuation) before comparing. If similarity > 85%, it flags for manual review instead of auto-skipping.

Risk 3: API Costs Spiral

What could go wrong: The agent finds 50 papers, calls Claude API 50 times, and I get a \$25 bill. **Guardrail:** Hard limit: max 10 papers processed per run. Cost estimate logged before any API call. If estimated cost > \$1, the agent pauses and asks for approval.

Risk 4: Agent Downloads Malicious PDF

What could go wrong: A paper title looks legitimate but the PDF is hosted on a malicious domain. **Guardrail:** Only download from trusted sources (arxiv.org, semanticscholar.org, openreview.net). Verify HTTPS certificate. Max file size 50MB. Scan with local antivirus before parsing.

Risk 5: Agent Overwrites My Existing Work

What could go wrong: A bug causes the agent to write to the wrong folder or overwrite an existing note. **Guardrail:** Write-only to research-notes/ and papers/. Never touch work/notebooks/, data/, or outputs/. File naming includes timestamp. Existing files are read-only to the agent.

Risk 6: I Stop Reading and Just Trust the Agent

What could go wrong: The agent becomes so good at summarizing that I stop opening the PDFs. I lose the ability to critically evaluate papers. **Guardrail:** The digest explicitly says: “These are starting points, not replacements. Open the PDF for the full method and limitations.” The agent is a scout, not a reader.

7. Build Platform and Justification

Chosen Platform: Claude Desktop + MCP + Local Python Scripts

How it works:

- **Claude Desktop** as the orchestrator (the “brain”)
- **MCP servers** for tool access (Filesystem, Web Search, PDF parsing)
- **Local Python scripts** for the heavy lifting (arXiv API calls, PDF download, text extraction)
- **Claude Project** loaded with the system prompt and identity kit

Why this platform:

1. **Free path exists:** arXiv API is free. Semantic Scholar API is free. Filesystem MCP is free. Claude Desktop is free. The only paid piece is Claude Pro (\$20/month) which I already have.
2. **I already built half of it:** My FL-04 workflow uses NotebookLM + Claude for research notes. This agent replaces the manual “find and upload” steps with automation, but keeps the synthesis step I already trust.
3. **MCP integration is proven:** I connected the Filesystem MCP in FL-05 and ran 3 successful tasks. I know it works.
4. **Debuggable:** If the agent fails, I can open Claude Desktop, see the tool calls, and trace exactly where it broke. No black-box SaaS.
5. **No vendor lock-in:** If Claude changes pricing, I can swap the LLM layer (use local Llama via Ollama) without rebuilding the pipeline.

Alternative Considered: n8n Cloud

Why I didn’t choose it:

- n8n Cloud free tier is limited (100 executions/month). My agent runs weekly = 52 executions/year, so the limit is fine.
- But n8n’s learning curve is steep for custom logic (duplicate detection, relevance scoring, voice formatting). I’d spend more time fighting the visual builder than writing the 200-line Python script.
- n8n is great for “when X happens, do Y” workflows. My agent needs conditional logic, loops, and LLM calls with structured prompts — easier in code than in nodes.
- **Verdict:** n8n is the right tool for business automation (CRM → Slack → Email). For research scouting with LLM synthesis, code + MCP is tighter.

Alternative Considered: Custom GPT (ChatGPT)

Why I didn't choose it:

- Custom GPTs require a paid ChatGPT Plus plan (\$20/month) — same cost as Claude Pro.
- Custom GPTs can't access my local file system. They can't read my existing notes or write new ones to my repo.
- They can use Actions (webhooks) but that requires building and hosting an API endpoint — more infrastructure than the agent itself.
- **Verdict:** Custom GPTs are good for chat-based assistants with web access. They're not good for agents that need to touch local files and run on a schedule.

8. Build Estimate: ~10 Hours

Task	Hours
Set up arXiv API wrapper + search logic	1.5
Set up Semantic Scholar API wrapper	1
Build duplicate detection (title normalization + similarity)	1.5
Build PDF download + text extraction pipeline	1.5
Write structured prompt for Claude synthesis	1
Build digest generation	1
Integrate with Filesystem MCP for read/write	1
Write eval cases + run tests	1
Debug edge cases (failures, rate limits, bad PDFs)	1
Total	~10.5 hours

Stretch goals (if under 10 hours):

- Add email digest delivery (send the Monday summary to my inbox)
- Add a “relevance score” that ranks papers by how closely they match my capstone

9. Honest Self-Check

Can I build this? Yes — the pieces are all things I've done or seen done. arXiv API is REST. PDF parsing is a library call. MCP is already connected. The hard part is the orchestration logic (when to retry, when to skip, when to ask me), not the tools.

Will I maintain it? Yes — because it saves me 45 minutes every week. The maintenance is: check the digest on Monday, spot-check one note, and tweak the search keywords if my capstone direction changes. That's 5 minutes of maintenance for 45 minutes of savings.

What if it fails? The fallback is my FL-04 manual workflow. If the agent breaks, I revert to “find papers on arXiv, paste into NotebookLM, format in Claude.” The agent is an upgrade, not a dependency.

Submitted to the General AI Fluency track thread.